

Lab 0 : Projet SmartCity – Visualisation de données avec la stack ELK

Objectif du Lab 0

- Mettre en place un environnement complet basé sur la Stack ELK avec Docker
- Générer un jeu de données simulées sur la pollution dans plusieurs villes
- Déployer une première pipeline Logstash pour ingestion et transformation de données
- Comprendre l'interconnexion entre Logstash, Elasticsearch et Kibana
- Créer un premier tableau de bord Kibana basé sur des données environnementales

Structure du projet

```
smartcity-elk/  
├── elk/  
│   ├── docker-compose.yml  
│   └── logstash.conf  
├── data/  
│   ├── pollution_data.csv  
│   └── generate_pollution_data.py  
├── jenkins/  
└── docs/
```

Étape 1 – Création de la structure de dossiers

Sous Windows PowerShell ou terminal Linux :

```
mkdir smartcity-elk  
cd smartcity-elk  
mkdir elk data jenkins docs
```

Cela permet d'organiser le projet dès le départ.

Étape 2 – Génération des données simulées de pollution

Script Python de génération de données

Script Python qui crée un fichier CSV de données simulées sur 4 villes françaises avec 4 types de polluants.

```
import csv
from datetime import datetime, timedelta
import random

# Définition des villes et polluants
villes = ["Paris", "Lyon", "Marseille", "Toulouse"]
polluants = ["NO2", "O3", "PM10", "SO2"]
start_date = datetime(2023, 1, 1)

# Création du fichier CSV
with open("pollution_data.csv", mode="w", newline="") as file:
    writer = csv.writer(file)
    writer.writerow(["date", "ville", "polluant", "valeur"])
    for i in range(100): # 100 jours
        date = (start_date + timedelta(days=i)).strftime("%Y-%m-%d")
        for ville in villes:
            for polluant in polluants:
                valeur = round(random.uniform(10, 100), 2)
                writer.writerow([date, ville, polluant, valeur])
```

Alternative avec Mockaroo

Si tu ne veux pas utiliser Python, tu peux aussi générer des données aléatoires directement depuis <https://mockaroo.com> :

1. Crée un schéma avec les colonnes suivantes : `date` , `ville` , `polluant` , `valeur`
2. Personnalise les types :
 - `date` → Type "Date"

- `ville` → Liste personnalisée (Paris, Lyon, Marseille, Toulouse)
- `polluant` → Liste personnalisée (NO2, O3, PM10, SO2)
- `valeur` → Type "Number" (min 10, max 100, precision 2)

3. Clique sur "Download Data" pour récupérer ton fichier CSV

4. Renomme-le en `pollution_data.csv` et place-le dans le dossier `data/`

Exécution du script

Dans le dossier `smartcity-elk/data`, exécute :

```
python generate_pollution_data.py
```

Vérifie que le fichier `pollution_data.csv` est bien créé.

Étape 3 – Rappels sur la Stack ELK

Composant	Rôle
Elasticsearch	Moteur NoSQL pour stocker, indexer, interroger des données en temps réel
Logstash	Ingestion, transformation et routage des données vers Elasticsearch
Kibana	Interface de visualisation connectée à Elasticsearch pour créer des tableaux de bord

Étape 4 – Configuration de Docker Compose et Logstash

Fichier `elk/docker-compose.yml`

Ce fichier permet de lancer en une seule commande tous les services nécessaires à notre environnement SmartCity avec la Stack ELK. Il définit les images Docker à utiliser, les ports exposés pour interagir avec les services, et les volumes pour échanger des fichiers entre le système hôte et les conteneurs.

Chaque section correspond à un conteneur indépendant mais interconnecté via un réseau Docker virtuel nommé `elk`. Cela garantit que les conteneurs peuvent communiquer entre eux tout en restant isolés du système hôte.

version: '3.8'

services:

elasticsearch:

Utilise l'image officielle d'Elasticsearch version 7.17.0

image: docker.elastic.co/elasticsearch/elasticsearch:7.17.0

environment:

Configure Elasticsearch en mode noeud unique (utile pour le dev)

- discovery.type=single-node

Désactive la sécurité (authentification, SSL) pour simplifier l'accès en local

- xpack.security.enabled=false

ports:

- "9200:9200"

networks:

- elk

kibana:

Interface graphique pour explorer les données stockées dans Elasticsearch

image: docker.elastic.co/kibana/kibana:7.17.0

ports:

- "5601:5601"

depends_on:

- elasticsearch

networks:

- elk

logstash:

Utilisé pour lire, transformer et envoyer les données vers Elasticsearch

image: docker.elastic.co/logstash/logstash:7.17.0

volumes:

- ./logstash.conf:/usr/share/logstash/pipeline/logstash.conf

- ../data/pollution_data.csv:/usr/share/logstash/pollution_data.csv

ports:

- "5044:5044"

depends_on:

- elasticsearch

```
networks:
```

```
- elk
```

```
networks:
```

```
elk:
```

```
# Réseau virtuel interne entre Elasticsearch, Logstash et Kibana
```

```
driver: bridge
```

Fichier `elk/logstash.conf`

Ce fichier contient les instructions que Logstash va suivre pour traiter les données de pollution :

- Lire le fichier CSV généré dans le dossier `data`
- Transformer chaque ligne du fichier en un document JSON structuré
- Ajouter des champs supplémentaires utiles comme la date au bon format
- Envoyer chaque document dans un index Elasticsearch nommé dynamiquement en fonction de la date

Voici le contenu détaillé et commenté :

```
input {
  # Spécifie que l'entrée de données est un fichier CSV local
  # "start_position" permet de lire le fichier depuis le début même s'il existe déjà
  # "sincedb_path" désactive le suivi d'état pour relire à chaque redémarrage
  file {
    path => "/usr/share/logstash/pollution_data.csv"
    start_position => "beginning"
    sincedb_path => "/dev/null"
  }
}

filter {
  # Le bloc 'csv' permet de transformer chaque ligne du fichier CSV en un document avec des champs nommés
  csv {
```

```

separator ⇒ ","
columns ⇒ ["date", "ville", "polluant", "valeur"]
}
mutate {
  # Convertit le champ "valeur" (au format texte) en nombre flottant
  convert ⇒ { "valeur" ⇒ "float" }
}
date {
  # Transforme le champ "date" (au format YYYY-MM-DD) en champ Elasticsearch "@timestamp"
  match ⇒ ["date", "YYYY-MM-dd"]
  target ⇒ "@timestamp"
}
mutate {
  # Ajoute dynamiquement un nom d'index basé sur la date du jour
  add_field ⇒ { "[@metadata][index_name]" ⇒ "%{+YYYY.MM.dd}" }
}
}

output {
  # Envoie les données transformées vers Elasticsearch
  elasticsearch {
    hosts ⇒ ["http://elasticsearch:9200"]
    index ⇒ "pollution-%{[@metadata][index_name]}"
  }

  # Affiche le contenu JSON final dans la console pour vérifier les transformations
  stdout { codec ⇒ rubydebug }
}

```

Étape 5 – Lancement de la stack ELK

Dans le terminal :

```

cd smartcity-elk/elk
docker-compose up -d

```

Vérification du bon fonctionnement

```
docker-compose ps
docker-compose logs -f logstash
```

Étape 6 – Accès aux interfaces des services

- **Kibana** : <http://localhost:5601>
- **Elasticsearch** : <http://localhost:9200>

Étape 7 – Visualisation des données dans Kibana

Après avoir lancé tous les conteneurs, Kibana est accessible via le navigateur à l'adresse : <http://localhost:5601>

Voici comment configurer l'affichage des données dans Kibana étape par étape :

1. Dans la barre latérale gauche de Kibana, clique sur **"Management"** (icône engrenage).
2. Sélectionne ensuite **"Stack Management"**, puis **"Index Patterns"** (ou "Index Management" selon la version).
3. Clique sur le bouton **"Create Index Pattern"**.
4. Dans le champ de nom d'index, saisis : `pollution-*` pour cibler tous les index créés par Logstash.
5. Clique sur "Next step", puis choisis le champ temporel **@timestamp** (généré automatiquement à partir du champ `date`).
6. Valide en cliquant sur **"Create index pattern"**.

Tu peux maintenant aller dans **"Discover"** dans le menu de gauche. Tu devrais voir s'afficher les données de pollution ligne par ligne (chaque document JSON).

1. Crée un Dashboard personnalisé :
 - Va dans le menu "Dashboard" > "Create new Dashboard"
 - Ajoute des visualisations comme :
 -  Histogramme : agrège les valeurs par date

-  Pie chart : pourcentage des types de polluants
-  Filtre interactif : filtrage par ville ou par polluant

Ces visualisations permettront de mieux analyser la pollution urbaine simulée dans un contexte SmartCity. Créer un Dashboard :

- Histogramme : valeur moyenne par jour
- Diagramme circulaire : proportion des polluants
- Filtres : par ville ou période

Conclusion – Résumé des Objectifs Atteints

Ce Lab 0 t'a permis de :

- Comprendre le rôle de chaque composant de la stack ELK
- Générer un jeu de données environnementales réalistes
- Déployer une infrastructure complète avec Docker
- Créer une pipeline Logstash pour traiter ces données
- Visualiser les résultats dynamiquement via Kibana

Ces étapes couvrent entièrement les objectifs fixés au début du lab et te donnent une base solide pour la suite du projet SmartCity.

👉 Ce Lab 0 permet d'installer tout l'environnement, comprendre le fonctionnement global, et valider un premier flux de bout en bout. Le prochain Lab ira plus loin sur l'automatisation CI/CD, les alertes et la sécurisation.